

Prova 1 – Inteligência Artificial: Algoritmos de Busca e Agentes Inteligentes

Gabriel Souza Baggio¹

¹Centro de Tecnologia – Universidade Federal de Santa Maria (UFSM)
Santa Maria – RS – Brasil

1. Questão 01 – Busca em amplitude/largura, Busca em profundidade e Busca iterativa em profundidade

1.1. Formulação como Problema de Busca em Espaços de Estados

- a) **Definição formal do estado:** Um estado corresponde à sala em que o agente se encontra no momento. O conjunto de todos os estados possíveis é $S = \{A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S\}$, totalizando 19 salas.
- b) **Estado inicial:** $s_0 = A$
- c) **Teste de objetivo:** O teste de objetivo verifica se o estado atual é igual ao estado objetivo: $\text{objetivo}(s) = (s == S)$.
- d) **Função sucessora:** A função sucessora retorna a lista de salas diretamente conectadas à sala s , na ordem definida pelo grafo.
- e) **Função de custo:** Cada movimento entre salas adjacentes possui custo unitário igual a 1. O custo total de um caminho com n movimentos é $g = n$. Portanto, o custo do caminho $A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$ é $g = 4$.
- f) **Representação em Python:** O código `q1.py` contém a representação criada.

1.2. Algoritmos de Busca

Esta subseção mostrará o desempenho dos algoritmos de busca em amplitude, busca em profundidade e busca em profundidade iterativa, avaliando quantidade de nós gerados, qualidade da resposta encontrada, além do entendimento acerca dos algoritmos.

1.2.1. Busca em Amplitude (BFS)

- a) **Ordem completa de expansão dos nós:** Tabela 1 exibe a ordem de expansão dos nós na segunda coluna.
- b) **Conteúdo da fronteira após cada expansão:** Tabela 1 mostra o conteúdo da fronteira para cada expansão.
- c) **Árvore parcial de busca** Figura 1 mostra o desenho à mão das árvores parciais (seguindo a ordem de expansão) e árvore final após todas expansões.
- d) **Caminho solução:** $A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$
- e) **Profundidade da solução:** $d = 4$
- f) **Custo da solução:** $g = 4$
- g) **Nós gerados:** 19
- h) **Nós expandidos:** 15 (A até O; S é detectado ao ser gerado, antes de ser expandido)

Table 1. Busca em Amplitude (BFS)

Passo	Nó expandido	Conteúdo da fronteira
1	A	[B, C, D]
2	B	[C, D, E, F]
3	C	[D, E, F, G, H]
4	D	[E, F, G, H, I]
5	E	[F, G, H, I, J]
6	F	[G, H, I, J, K, L]
7	G	[H, I, J, K, L, M]
8	H	[I, J, K, L, M, N, O]
9	I	[J, K, L, M, N, O, P]
10	J	[K, L, M, N, O, P]
11	K	[L, M, N, O, P, Q]
12	L	[M, N, O, P, Q]
13	M	[N, O, P, Q, R]
14	N	[O, P, Q, R]
15	O	[P, Q, R, S]
16	S*	objetivo encontrado

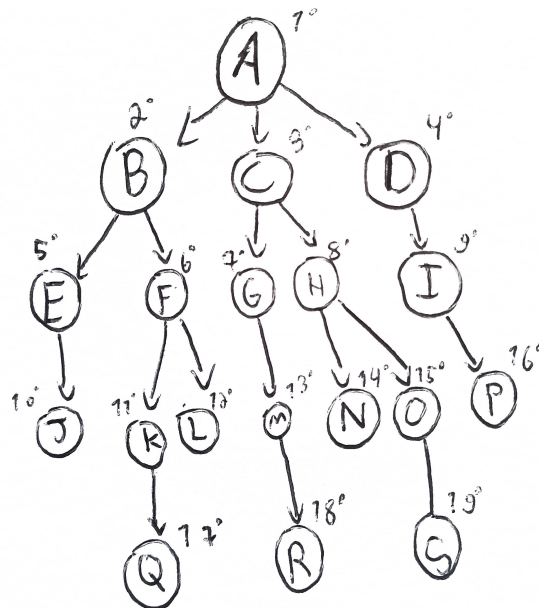


Figure 1. Busca em Amplitude Desenho

1.2.2. Busca em Profundidade (DFS)

- Ordem completa de expansão dos nós:** Tabela 2 exibe a ordem de expansão dos nós na segunda coluna.
- Conteúdo da fronteira após cada expansão:** Tabela 2 mostra o conteúdo da fronteira para cada expansão.
- Árvore parcial de busca:** Figura 2 mostra o desenho à mão das árvores parciais (seguindo a ordem de expansão) e árvore final após todas expansões.

- d) **Caminho solução:** $A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$
e) **Profundidade da solução:** $d = 4$
f) **Custo da solução:** $g = 4$
g) **Nós gerados:** 16
h) **Nós expandidos:** 15 (A até O; S é detectado ao ser gerado)

Table 2. Busca em Profundidade (BFS)

Passo	Nó expandido	Conteúdo da fronteira (pilha, topo à esquerda)
1	A	[B, C, D]
2	B	[E, F, C, D]
3	E	[J, F, C, D]
4	J	[F, C, D]
5	F	[K, L, C, D]
6	K	[Q, L, C, D]
7	Q	[L, C, D]
8	L	[C, D]
9	C	[G, H, D]
10	G	[M, H, D]
11	M	[R, H, D]
12	R	[H, D]
13	H	[N, O, D]
14	N	[O, D]
15	O	[S, D]
16	S*	objetivo encontrado

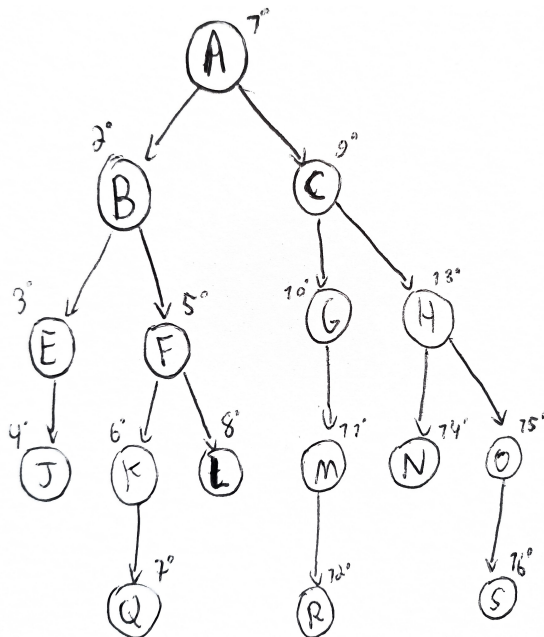


Figure 2. Busca em Profundidade Desenho

1.2.3. Busca Iterativa em Profundidade (IDS)

a) **Ordem completa de expansão dos nós:** As tabelas no item (b) mostram a ordem de expansão dos nós em cada iteração.

b) **Conteúdo da fronteira após cada expansão:**

Limite = 0:

Passo	Nó expandido	Fronteira
1	A	[] (filhos cortados pelo limite)

Limite = 1:

Passo	Nó expandido	Fronteira
1	A	[B, C, D]
2	B	[C, D] (filhos de B cortados)
3	C	[D] (filhos de C cortados)
4	D	[] (filhos de D cortados)

Limite = 2:

Passo	Nó expandido	Fronteira
1	A	[B, C, D]
2	B	[E, F, C, D]
3	E	[F, C, D] (filhos de E cortados)
4	F	[C, D] (filhos de F cortados)
5	C	[G, H, D]
6	G	[H, D] (filhos de G cortados)
7	H	[D] (filhos de H cortados)
8	D	[I]
9	I	[] (filhos de I cortados)

Limite = 3:

Passo	Nó expandido	Fronteira
1	A	[B, C, D]
2	B	[E, F, C, D]
3	E	[J, F, C, D]
4	J	[F, C, D]
5	F	[K, L, C, D]
6	K	[L, C, D] (filhos de K cortados)
7	L	[C, D]
8	C	[G, H, D]
9	G	[M, H, D]
10	M	[H, D] (filhos de M cortados)
11	H	[N, O, D]
12	N	[O, D]
13	O	[D] (filhos de O cortados)
14	D	[I]
15	I	[P]
16	P	[]

Limite = 4 (iteração final):

Passo	Nó expandido	Fronteira
1	A	[B, C, D]
2	B	[E, F, C, D]
3	E	[J, F, C, D]
4	J	[F, C, D]
5	F	[K, L, C, D]
6	K	[Q, L, C, D]
7	Q	[L, C, D]
8	L	[C, D]
9	C	[G, H, D]
10	G	[M, H, D]
11	M	[R, H, D]
12	R	[H, D]
13	H	[N, O, D]
14	N	[O, D]
15	O	[S, D]
16	S*	objetivo encontrado

c) **Árvore parcial de busca:** Figura 3 mostra o desenho à mão das árvores parciais (seguindo a ordem de expansão) e árvore final após todas expansões, para cada uma das iterações.

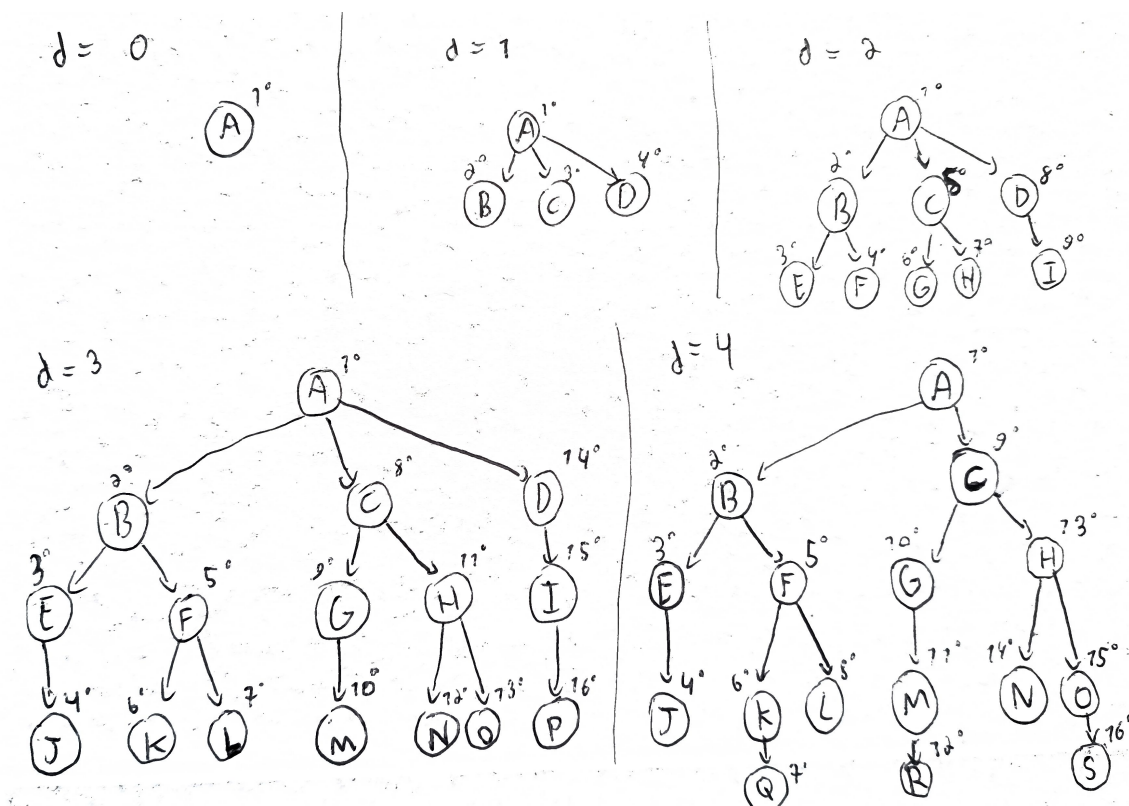


Figure 3. Busca em Profundidade Iterativa Desenho

d) **Caminho solução:** $A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$

e) **Profundidade da solução:** $d = 4$

f) **Custo da solução:** $g = 4$

g) **Nós gerados (total acumulado em todas as iterações):**

- Limite 0: 1 nó gerado (A), 1 expandido
- Limite 1: 4 nós gerados, 4 expandidos
- Limite 2: 9 nós gerados, 9 expandidos
- Limite 3: 16 nós gerados, 16 expandidos
- Limite 4: 16 nós gerados, 15 expandidos (S encontrado ao ser gerado)
- **Total gerado: 46 nós**

h) **Nós expandidos** $1 + 4 + 9 + 16 + 15 = 45$ (total acumulado)

1.3. Experimento Adicional – Alteração da Ordem dos Sucessores

1.3.1. Modificações Realizadas

Foram alteradas as listas de sucessores de dois nós:

- **Nó H:** $N, O \rightarrow O, N$
- **Nó C:** $G, H \rightarrow H, G$

1.3.2. Análise das Questões Comparativas

- O caminho solução mudou?** Não, para nenhum dos três algoritmos. O caminho $A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$ com custo 4 é o único caminho de menor profundidade até S neste grafo, e todos os algoritmos o encontram independentemente da ordem dos sucessores.
- A quantidade de nós expandidos mudou?** Sim. A nova ordem de sucessores (H antes de G em C; O antes de N em H) faz a busca atingir S antes de explorar ramos mortos como $G \rightarrow M \rightarrow R$ e N , reduzindo o número de expansões. Para o BFS, ele reduz um pouco por conta de expandir o nó (O) antes, encontrando então o nó (S).
- Algum algoritmo foi mais sensível à ordem dos sucessores?** Sim. A DFS e a IDS são significativamente mais sensíveis, pois exploram um ramo até seu fim antes de retroceder; a ordem determina qual ramo é explorado primeiro, impactando diretamente o número de nós visitados. A BFS é mais insensível, pois expande todos os nós de um nível antes de avançar.
- Qual algoritmo apresentou maior consumo de memória?** A BFS, pois mantém na fila (fronteira) todos os nós gerados de um nível que ainda não foram expandidos. No pior caso, a fronteira cresce exponencialmente com a largura da árvore. A DFS e a IDS mantêm apenas o caminho atual na pilha, com consumo linear $O(d)$.
- Qual algoritmo encontrou a solução mais rapidamente?** No grafo original, BFS e DFS expandem o mesmo número de nós (15) antes de encontrar S. Com as alterações na ordem dos sucessores, a DFS encontra S mais rapidamente por explorar primeiro o ramo correto. Em geral, para este problema específico, a DFS (com ordem favorável) é a mais rápida; a IDS é a mais lenta em número total de expansões por repetir nós a cada iteração.

1.4. Comparação Final dos Algoritmos

Critério	BFS	DFS	IDS
a) Completude	Sim (em grafos finitos)	Sim (em grafos finitos sem ciclos; pode falhar em grafos infinitos)	Sim (em grafos finitos)
b) Otimalidade	Sim (custo unitário)	Não (pode encontrar caminho mais longo)	Sim (custo unitário)
c) Complexidade de tempo	$O(b^d)$	$O(b^m)$	$O(b^d)$
d) Complexidade de espaço	$O(b^d)$	$O(b \cdot m)$	$O(b \cdot d)$
e) Dependência da ordem dos sucessores	Baixa (ordem dentro do nível)	Alta (determina qual ramo é explorado)	Alta (igual à DFS em cada iteração)
f) Comportamento em árvores profundas	Inviável: memória cresce exponencialmente	Eficiente em memória, mas pode explorar ramos muito profundos sem encontrar solução	Equilibrado: eficiente em memória e garante encontrar a solução ótima
g) Adequação para problemas grandes	Limitada pela memória	Adequada em memória, mas sem garantia de otimalidade	Melhor combinação: memória de DFS com otimalidade de BFS

Nota: b = fator de ramificação, d = profundidade da solução ótima, m = profundidade máxima da árvore.

1.5. Código Python

Olhar arquivo `q1.py` para ver a implementação.

2. Questão 02 – Busca Gulosa pela Melhor Escolha e A*

- a) **Representação dos Estados:** Cada estado representa a posição atual do agente no grafo. O conjunto completo de estados é $S = \{A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T\}$.
- b) **Estado Inicial:** O estado inicial é o nó **A**, posição de partida do agente.
- c) **Teste de Objetivo:** O teste de objetivo verifica se o nó atual é igual a **T**.
- d) **Função Sucessora:** A função sucessora retorna os pares (nó vizinho, custo da aresta) para cada nó n , conforme definido pelas arestas do grafo.
- e) **Função de Custo:** O custo de uma aresta (u, v) é o valor numérico positivo associado a ela no grafo. O custo acumulado $g(n)$ de um caminho da raiz até o nó n é a soma dos custos de todas as arestas percorridas.
- f) **Interpretação da Heurística $h(n)$:** A heurística $h(n)$ representa uma **estimativa do custo restante** para alcançar o objetivo T a partir do nó n . Ela deve ser *admissível*, ou seja, nunca superestimar o custo real, para garantir a otimalidade do A*. Por exemplo, $h(A) = 10$ significa que se estima que o caminho mínimo de A até T custa pelo menos 10 unidades.

2.1. Busca Gulosa pela Melhor Escolha (Greedy Best-First Search)

O algoritmo seleciona em cada passo o nó da fronteira com menor $h(n)$, ignorando o custo acumulado $g(n)$.

- a) **Ordem de expansão dos nós:** Tabela 3 exibe a ordem de expansão dos nós na segunda coluna. A Figura 4 mostra visualmente a execução do algoritmo.
- b) **Conteúdo da fronteira após cada expansão:** Tabela 3 mostra o conteúdo da fronteira para cada expansão. A Figura 4 mostra visualmente a execução do algoritmo.
- c) **Valores Heurísticos Utilizados:** A Tabela 3 mostra o $h(n)$ utilizado a cada expansão. A Figura 4 mostra visualmente a execução do algoritmo.

Table 3. Execução da Busca Gulosa

Passo	Nó expandido	Fronteira (nó: h)	h(n) usado
1	A	B:8, D:9, C:7	10
2	C	B:8, D:9, G:6, H:4	7
3	H	B:8, D:9, G:6, N:4, O:1	4
4	O	B:8, D:9, G:6, N:4, T:0	1
5	T	(objetivo encontrado)	0

- d) **Caminho Solução Encontrado:** $A \rightarrow C \rightarrow H \rightarrow O \rightarrow T$
- e) **Custo Final da Solução:** $g(T) = 4 + 6 + 4 + 5 = 19$
- f) **Quantidade de Nós Gerados:** 9
- g) **Quantidade de Nós Expandidos:** 5

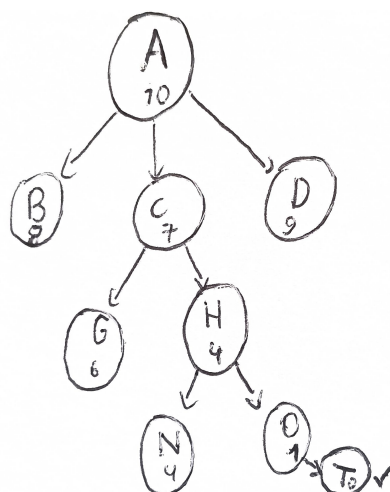


Figure 4. Busca Gulosa para Melhor Escolha Desenho

2.2. Busca A*

O A* seleciona o nó com menor $f(n) = g(n) + h(n)$. A Figura 5 e Tabela 4 mostram, em conjunto, para cada expansão: (a) nó expandido, (b) valor de $g(n)$, (c) valor de $h(n)$, (d) valor de $f(n)$, (e) conteúdo da fronteira ordenada e (f) caminho parcial até o nó.

Caminho final até o destino com A*: $A \rightarrow B \rightarrow F \rightarrow K \rightarrow R \rightarrow T$

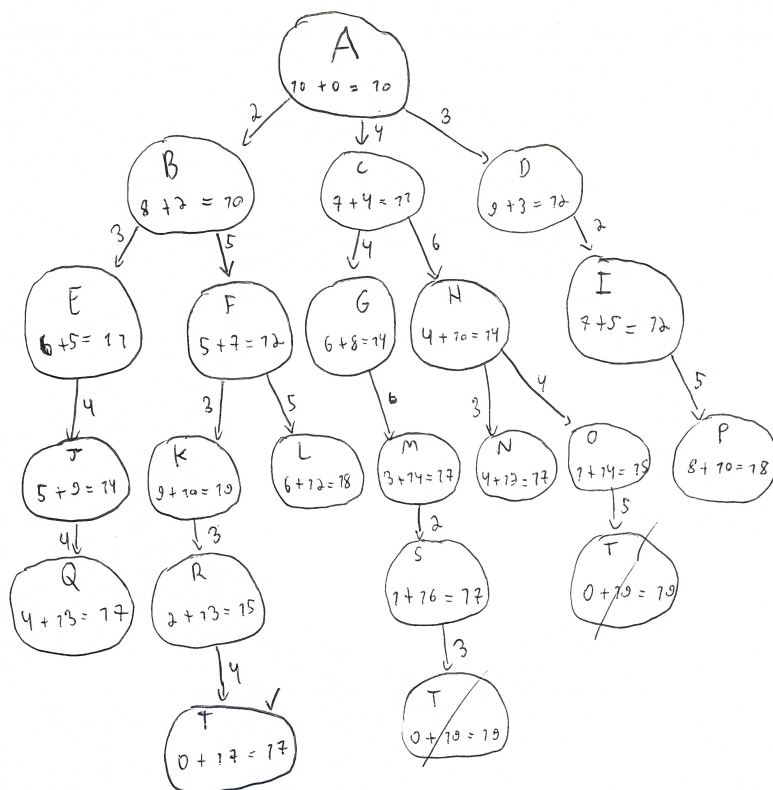


Figure 5. Busca A* Desenho

P	Nó expandido	g(n)	h(n)	f(n)	Fronteira ordenada por f	Caminho parcial
1	A	0	10	10	–	A
2	B	2	8	10	C:11, D:12	A → B
3	C	4	7	11	E:11, D:12, F:12	A → C
4	E	5	6	11	D:12, F:12, G:14, H:14	A → B → E
5	D	3	9	12	F:12, G:14, H:14, J:14	A → D
6	I	5	7	12	F:12, G:14, H:14, J:14	A → D → I
7	F	7	5	12	G:14, H:14, J:14, P:18	A → B → F
8	K	10	3	13	G:14, H:14, J:14, L:18, P:18	A → B → F → K
9	G	8	6	14	H:14, J:14, R:15, L:18, P:18	A → C → G
10	J	9	5	14	H:14, R:15, M:17, L:18, P:18	A → B → E → J
11	H	10	4	14	R:15, M:17, Q:17, L:18, P:18	A → C → H
12	R	13	2	15	O:15, M:17, N:17, Q:17, L:18, P:18	A → B → F → K → R
13	O	14	1	15	M:17, N:17, Q:17, T:17, L:18, P:18	A → C → H → O
14	Q	13	4	17	M:17, N:17, T:17, L:18, P:18	A → B → E → J → Q
15	N	13	4	17	M:17, T:17, L:18, P:18	A → C → H → N
16	M	14	3	17	T:17, L:18, P:18	A → C → G → M
17	S	16	1	17	T:17, L:18, P:18	A → C → G → M → S
18	T	17	0	17	L:18, P:18	A → B → F → K → R → T

Table 4. Execução do Algoritmo A*

2.3. Modificação das Heurísticas e Análise do Impacto

2.3.1. Modificações Realizadas

Foram alterados os valores heurísticos dos nós **O**, **S** e **R**:

Table 5. Heurísticas originais vs. modificadas

Nó	h original	h modificada
O	1	10
S	1	10
R	2	1

Note que neste caso a heurística **deixou de ser admissível**. Com isso, podemos testar o impacto no algoritmo A*, visto que ele pode encontrar uma solução não ótima agora.

2.3.2. Análise das Questões Comparativas

a) O caminho solução mudou?

Busca Gulosa: Sim. Com $h(O) = 10$, o caminho via O deixa de ser atrativo e a busca gulosa tende a seguir outro ramo, alterando o caminho encontrado.

A*: Não. Nesse caso, o A* encontrou o mesmo caminho, mesmo com uma heurística não admissível.

b) O custo da solução mudou?

Busca Gulosa: Sim, pois a busca gulosa não garante otimalidade e segue caminhos diferentes conforme a heurística.

A*: Não. Nesse caso, como o caminho do A* não tinha mudado, então o seu custo também não é alterado.

c) A busca gulosa encontrou solução ótima?

Não. A busca gulosa encontrou um resultado com custo $g = 19$, enquanto que o caminho ótimo tem custo $g = 17$.

d) **O A* encontrou solução ótima?**

Sim. Ele continuou encontrando a solução ótima, mesmo com heurística não admissível (sorte, nesse caso).

e) **Qual algoritmo expandiu menos nós?**

A **busca gulosa** expandiu menos nós em ambas as configurações, pois abandona ramos assim que encontra um caminho até T, sem verificar se é ótimo.

f) **Qual algoritmo foi mais sensível à heurística?**

A **busca gulosa** é mais sensível, pois toma decisões baseadas exclusivamente em $h(n)$. Pequenas alterações nos valores heurísticos mudam completamente o caminho encontrado e o custo final. Nesse caso, o A* não teve nenhuma alteração, mesmo alterando para uma heurística não admissível.

2.4. Comparação entre Greedy Best-First Search e A*

Table 6. Comparação entre Busca Gulosa e A*

Critério	Greedy Best-First	A*
a) Completude	Completo em espaços finitos sem ciclos; pode falhar em grafos com ciclos sem detecção de nós visitados.	Completo em grafos finitos com custos positivos.
b) Otimalidade	Não garante solução ótima.	Garante solução ótima se $h(n)$ for admissível (e consistente em grafos).
c) Memória	Menor uso de memória; mantém apenas a fronteira.	Maior uso: armazena todos os nós gerados para comparar $f(n)$.
d) Dependência da Heurística	Alta dependência: decisões são tomadas <i>somente</i> com base em $h(n)$. Heurísticas ruins levam a soluções muito ruins.	Moderada: $h(n)$ influencia a eficiência, mas $g(n)$ garante consistência. Heurísticas ruins aumentam nós expandidos.
e) Custo Computacional	Baixo: menos nós expandidos, mas sem garantia de qualidade.	Mais alto: expande mais nós para garantir otimalidade.
f) Qualidade das Soluções	Variável; pode ser ótima ou muito subótima.	Ótima (dado que a heurística seja admissível).
g) Grafos Grandes	Mais adequado quando a memória é limitada e uma solução razoável é suficiente.	Pode ser inviável sem podas, pois o espaço de estados cresce exponencialmente; variantes como IDA* mitigam isso.

2.5. Código Python

Olhar arquivo `q2.py` para ver a implementação.

3. Questão 03 – Busca Local em Problema das N-Rainhas

3.1. Modelagem como Problema de Busca Local

- a) **Representação do Estado:** Cada estado é representado por um vetor de 8 inteiros:

$$s = [s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8]$$

onde $s_i \in \{1, 2, \dots, 8\}$ indica a linha ocupada pela rainha posicionada na coluna i . Essa representação garante implicitamente que existe exatamente uma rainha por coluna.

- b) **Definição de Vizinho:** Um estado vizinho de s é obtido movendo **exatamente uma rainha** para outra linha da **mesma coluna**. Formalmente, s' é vizinho de s se e somente se existe exatamente uma coluna $c \in \{1, \dots, 8\}$ tal que $s'_c \neq s_c$, e $s'_i = s_i$ para todo $i \neq c$.
- c) **Função de Avaliação:** A função heurística $h(s)$ conta o número total de pares de rainhas em conflito. Duas rainhas conflitam quando estão na mesma linha ($s_i = s_j$) ou na mesma diagonal ($|s_i - s_j| = |i - j|$).
- d) **Critério de Parada:** **Solução global:** $h(s) = 0$; ou **Estagnação:** nenhum vizinho possui h estritamente menor que o estado atual (máximo local ou platô).
- e) **Interpretação da Superfície de Busca:** A superfície de busca é um espaço de dimensão 8, onde cada ponto representa uma configuração das rainhas e a altitude representa $h(s)$. O objetivo é minimizar h , encontrando o “vale mais profundo” com altitude zero.

3.2. Problemas de Estagnação

O algoritmo de Hill-Climbing possui algumas complicações:

- a) **Máximos locais:** estados em que todo vizinho tem h maior, mas ainda $h > 0$. O Hill-Climbing para sem encontrar a solução.
- b) **Platôs:** regiões planas onde múltiplos estados adjacentes compartilham o mesmo valor de h . O Hill-Climbing não tem direção preferencial de melhora.
- c) **Picos estreitos:** vales de boa qualidade rodeados por estados piores, difíceis de alcançar por exploração gulosa.

3.3. Resolução Manual com Hill-Climbing

O algoritmo parte de $s_0 = [1, 1, 1, 1, 1, 1, 1, 1]$, com $h(s_0) = 28$ (todos os $\binom{8}{2} = 28$ pares em conflito, pois todas as rainhas estão na mesma linha). A cada iteração, todos os 56 vizinhos são avaliados e o de menor h é escolhido.

3.3.1. Detalhamento de Cada Iteração

Iteração 0 – Estado: [1,1,1,1,1,1,1], $h = 28$

- a) **Estado atual:** [1, 1, 1, 1, 1, 1, 1], $h = 28$.
b) **Melhor vizinho escolhido:** [1, 8, 1, 1, 1, 1, 1], $h = 21$.

Table 7. Top 5 vizinhos na Iteração 0

Vizinho	$h(s)$	Obs.
[1, 8, 1, 1, 1, 1, 1]	21	Escolhido
[1, 1, 7, 1, 1, 1, 1]	21	
[1, 1, 8, 1, 1, 1, 1]	21	
[1, 1, 1, 6, 1, 1, 1]	21	
[1, 1, 1, 7, 1, 1, 1]	21	

Iteração 1 – Estado: [1,8,1,1,1,1,1], $h = 21$

- a) **Estado atual:** [1, 8, 1, 1, 1, 1, 1], $h = 21$.
b) **Melhor vizinho escolhido:** [2, 8, 1, 1, 1, 1, 1], $h = 15$.

Table 8. Top 5 vizinhos na Iteração 1

Vizinho	$h(s)$	Obs.
[2, 8, 1, 1, 1, 1, 1]	15	Escolhido
[1, 8, 1, 7, 1, 1, 1]	15	
[1, 8, 1, 1, 6, 1, 1]	15	
[1, 8, 1, 1, 7, 1, 1]	15	
[1, 8, 1, 1, 1, 5, 1, 1]	15	

Iteração 2 – Estado: [2,8,1,1,1,1,1], $h = 15$

- a) **Estado atual:** [2, 8, 1, 1, 1, 1, 1], $h = 15$.
b) **Melhor vizinho escolhido:** [2, 8, 1, 7, 1, 1, 1], $h = 10$.

Table 9. Top 5 vizinhos na Iteração 2

Vizinho	$h(s)$	Obs.
[2, 8, 1, 7, 1, 1, 1]	10	Escolhido
[2, 8, 1, 1, 7, 1, 1]	10	
[2, 8, 1, 1, 1, 5, 1, 1]	10	
[2, 8, 1, 1, 1, 6, 1, 1]	10	
[2, 8, 1, 1, 1, 1, 6, 1]	10	

Iteração 3 – Estado: [2,8,1,7,1,1,1,1], $h = 10$

- a) **Estado atual:** [2, 8, 1, 7, 1, 1, 1, 1], $h = 10$.
b) **Melhor vizinho escolhido:** [2, 8, 1, 7, 1, 6, 1, 1], $h = 6$.

Table 10. Top 5 vizinhos na Iteração 3

Vizinho	$h(s)$	Obs.
[2, 8, 1, 7, 1, 6, 1, 1]	6	Escolhido
[2, 8, 1, 7, 1, 1, 6, 1]	6	
[2, 8, 1, 7, 1, 1, 1, 5]	6	
[2, 8, 2, 7, 1, 1, 1, 1]	7	
[2, 8, 3, 7, 1, 1, 1, 1]	7	

Iteração 4 – Estado: [2,8,1,7,1,6,1,1], $h = 6$

- a) **Estado atual:** [2, 8, 1, 7, 1, 6, 1, 1], $h = 6$.
b) **Melhor vizinho escolhido:** [2, 8, 1, 7, 1, 6, 1, 5], $h = 3$.

Table 11. Top 5 vizinhos na Iteração 4

Vizinho	$h(s)$	Obs.
[2, 8, 1, 7, 1, 6, 1, 5]	3	Escolhido
[2, 8, 2, 7, 1, 6, 1, 1]	4	
[2, 8, 4, 7, 1, 6, 1, 1]	4	
[2, 8, 5, 7, 1, 6, 1, 1]	4	
[2, 8, 1, 7, 2, 6, 1, 1]	4	

Iteração 5 – Estado: [2,8,1,7,1,6,1,5], $h = 3$

- a) **Estado atual:** [2, 8, 1, 7, 1, 6, 1, 5], $h = 3$.
b) **Melhor vizinho escolhido:** [2, 8, 1, 7, 4, 6, 1, 5], $h = 1$.

Table 12. Top 5 vizinhos na Iteração 5

Vizinho	$h(s)$	Obs.
[2, 8, 1, 7, 4, 6, 1, 5]	1	Escolhido
[2, 8, 2, 7, 1, 6, 1, 5]	2	
[2, 8, 4, 7, 1, 6, 1, 5]	2	
[2, 8, 1, 7, 1, 6, 2, 5]	2	
[2, 8, 3, 7, 1, 6, 1, 5]	3	

Iteração 6 – Estado: [2,8,1,7,4,6,1,5], $h = 1$

- a) **Estado atual:** [2, 8, 1, 7, 4, 6, 1, 5], $h = 1$.
- b) **Melhor vizinho:** [2, 8, 3, 7, 4, 6, 1, 5], $h = 1$.

Table 13. Top 5 vizinhos na Iteração 6

Vizinho	$h(s)$	Obs.
[2, 8, 3, 7, 4, 6, 1, 5]	1	Escolhido (desempate posicional)
[2, 8, 1, 7, 4, 6, 3, 5]	1	
[3, 8, 1, 7, 4, 6, 1, 5]	2	
[5, 8, 1, 7, 4, 6, 1, 5]	2	
[6, 8, 1, 7, 4, 6, 1, 5]	2	

3.3.2. Resultado Final

- a) **Estado final:** [2, 8, 1, 7, 4, 6, 1, 5], $h = 1$.
- b) **Total de iterações:** 7.
- c) **Motivo de parada:** platô (nenhum vizinho possui $h < 1$).
- d) **Solução encontrada:** não ($h \neq 0$).

3.3.3. Identificação de Fenômenos de Busca

Na Iteração 6 ocorreu um **platô**: o estado atual tem $h = 1$ e os dois melhores vizinhos também têm $h = 1$. Nenhum movimento de uma única rainha resolve o conflito remanescente sem criar outro.

- a) **Por que ocorreu:** o único par de rainhas em conflito está posicionado de forma que qualquer movimento capaz de eliminar esse conflito introduz imediatamente um novo, mantendo $h = 1$ em toda a vizinhança acessível.
- b) **Como afetou a busca:** o Hill-Climbing convencional exige melhora estrita e para ao detectar o platô, sem atingir a solução global.
- c) **Como poderia ser evitado:** permitindo movimentos laterais (aceitar vizinhos com h igual), aplicando *random restart* ao detectar a estagnação.

3.4. Random Restart Hill-Climbing

O algoritmo é executado 20 vezes com estados iniciais gerados aleatoriamente.

Resultado encontrado ao rodar o algoritmo: 2 soluções encontradas em 20 execuções (taxa de sucesso de 10%), com média de 3,1 passos por execução. A taxa de sucesso reduzida evidencia a alta densidade de máximos locais e platôs no espaço de busca.

Table 14. Random Restart Hill-Climbing – 20 execuções

Execução	Estado inicial	Passos	h(s) final	Solução?
1	[2, 1, 5, 4, 4, 3, 2, 2]	4	1	NÃO
2	[7, 1, 1, 2, 4, 4, 1, 4]	5	0	SIM
3	[7, 4, 8, 5, 1, 3, 7, 6]	2	2	NÃO
4	[5, 3, 4, 6, 2, 2, 7, 2]	2	1	NÃO
5	[6, 6, 5, 1, 8, 2, 7, 2]	4	0	SIM
6	[5, 6, 4, 2, 1, 4, 5, 2]	3	1	NÃO
7	[4, 2, 7, 5, 8, 6, 3, 6]	1	2	NÃO
8	[6, 4, 5, 2, 3, 4, 3, 8]	3	1	NÃO
9	[7, 5, 4, 6, 1, 4, 1, 6]	4	1	NÃO
10	[7, 5, 2, 4, 6, 4, 8, 7]	2	2	NÃO
11	[8, 3, 5, 3, 4, 5, 7, 7]	4	1	NÃO
12	[6, 4, 3, 8, 2, 1, 2, 3]	3	1	NÃO
13	[3, 7, 2, 7, 7, 8, 5, 1]	2	2	NÃO
14	[2, 5, 6, 2, 5, 7, 3, 8]	3	2	NÃO
15	[1, 5, 3, 2, 5, 4, 3, 6]	3	2	NÃO
16	[3, 1, 6, 8, 1, 2, 6, 5]	2	1	NÃO
17	[4, 1, 4, 2, 2, 8, 2, 3]	3	1	NÃO
18	[3, 8, 3, 5, 7, 4, 4, 5]	3	1	NÃO
19	[7, 6, 8, 8, 2, 4, 4, 2]	4	2	NÃO
20	[6, 1, 4, 4, 1, 2, 1, 4]	5	1	NÃO

3.5. Simulated Annealing

3.5.1. Configuração do Algoritmo

- Valor inicial da temperatura:** $T_0 = 100,0$. Suficientemente alto para que, no início, quase qualquer movimento seja aceito, permitindo ampla exploração do espaço de busca.
- Política de resfriamento:** $T_{t+1} = \alpha \cdot T_t$ com $\alpha = 0,95$. A temperatura decresce 5% a cada iteração, convergindo suavemente de exploração (altas temperaturas) para exploração (baixas temperaturas). O critério de parada adicional é $T < 10^{-10}$ ou $h = 0$.

3.5.2. Exemplos de Movimentos Piores (ou iguais) Aceitos

A Tabela 15 apresenta casos em que o Simulated Annealing aceitou um estado com h maior ou igual ao atual durante a execução partindo de $[1, 1, 1, 1, 1, 1, 1, 1]$.

Movimentos com $\Delta E = 0$ (platô) são aceitos com $P = e^0 = 1,0$, diferenciando o Simulated Annealing do Hill-Climbing simples já em situações de platô, sem necessidade de temperatura elevada.

Table 15. Casos de movimentos piores aceitos pelo Simulated Annealing

Passo	h antes	h depois	ΔE	T	P(aceitar)
3	13	13	0	85,74	1,0000
4	13	14	1	81,45	0,9878
10	6	6	0	59,87	1,0000
11	6	9	3	56,88	0,9486
12	9	11	2	54,04	0,9637

3.5.3. Comparação com Hill-Climbing

Como teste, partindo de $[1, 1, 1, 1, 1, 1, 1, 1]$, o Simulated Annealing encontrou a solução $[3, 1, 7, 5, 8, 2, 4, 6]$ com $h = 0$ em 224 passos. Em 20 execuções com estados iniciais aleatórios, obtemos:

Table 16. Comparação entre Random Restart Hill-Climbing e Simulated Annealing (20 execuções)

Algoritmo	Soluções/20	Média de Passos
Hill-Climbing (Random Restart)	2	3,1
Simulated Annealing	13	357,8

- Qualidade:** o Simulated Annealing encontrou solução em 13 de 20 execuções (65%), contra 10% do Random Restart Hill-Climbing.
- Velocidade:** o Simulated Annealing usa mais passos por execução (~ 358 contra ~ 3), pois explora regiões piores antes de convergir.
- Movimentos piores aceitos:** a aceitação probabilística permite ao Simulated Annealing escapar de máximos locais e platôs onde o Hill-Climbing trava.
- Comparação geral:** o Simulated Annealing é mais lento, porém encontra soluções de qualidade significativamente superior ao Hill-Climbing simples.
- Quantidade de soluções válidas encontradas:** 13 de 20 execuções.

3.6. Comparação Geral entre os Algoritmos

Table 17. Comparação entre Hill-Climbing, Random Restart Hill-Climbing e Simulated Annealing

Critério	Hill-Climbing	Random Restart Hill-Climbing	Simulated Annealing
a) Qualidade das soluções	Baixa (preso em ótimos locais)	Moderada	Alta
b) Velocidade de convergência	Muito rápida (~ 7 passos)	Rápida por execução (~ 3 passos)	Lenta (~ 358 passos)
c) Sensibilidade ao estado inicial	Muito alta	Reduzida pelos reinícios	Moderada
d) Capacidade de escapar de máximos locais	Nula	Sim (via reinício)	Sim (via probabilidade)
e) Custo computacional	Mínimo	Proporcional ao número de reinícios	Alto
f) Estabilidade dos resultados	Alta (determinístico)	Moderada	Variável (depende de T_0 e α)

3.7. Código Python

Olhar arquivo `q3.py` para ver a implementação.

4. Questão 04 – CSP e Otimizações de Busca

4.1. Modelagem como CSP

- a) **Conjunto de Variáveis:** Cada turno de plantão é representado por uma variável:

$$X = \{T_1, T_2, T_3, T_4, T_5, T_6\}$$

- b) **Domínio de Cada Variável:** O domínio inicial de cada variável contém todos os médicos disponíveis. Após a aplicação das restrições unárias, o domínio de T_3 é reduzido:

$$D(T_1) = D(T_2) = D(T_4) = D(T_5) = D(T_6) = \{A, B, C, D\}$$

$$D(T_3) = \{B, C, D\}$$

- c) **Restrições Unárias:** O médico A não pode ser atribuído ao turno T_3 :

$$T_3 \neq A$$

- d) **Restrições Binárias:** O mesmo médico não pode trabalhar em turnos consecutivos. Para cada par (T_i, T_{i+1}) com $i \in \{1, \dots, 5\}$:

$$T_i \neq T_{i+1}$$

Gerando os pares: (T_1, T_2) , (T_2, T_3) , (T_3, T_4) , (T_4, T_5) , (T_5, T_6) .

Segunda restrição binária: Exclusão mútua de C em T_2 e T_5 . Equivale a: se $T_2 = C$ então $T_5 \neq C$, e vice-versa.

- e) **Restrições Globais:** Presença de B em T_1 ou T_2 :

$$T_1 = B \text{ ou } T_2 = B$$

Segunda restrição global: o médico D pode assumir no máximo dois turnos.

- f) **Grafo de Restrições:** O grafo de restrições possui as variáveis $T_1, \dots, T_6$ como nós. As arestas representam as restrições binárias:

- **Arestas de não-consecutividade** (cadeia linear): $T_1 - T_2 - T_3 - T_4 - T_5 - T_6$
- **Aresta de exclusão de C:** $T_2 - T_5$ (restrição condicional sobre o valor C)

As restrições globais (presença de B , limite de D) não possuem representação como aresta simples e são modeladas como restrições de ordem superior.

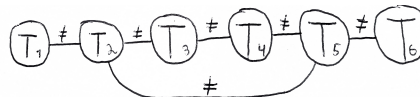


Figure 6. Grafo de Restrições CSP

4.2. Resolução por Backtracking Manual

A busca percorre as variáveis na ordem fixa $T_1, T_2, T_3, T_4, T_5, T_6$, tentando valores na ordem A, B, C, D .

4.2.1. Tabela de Passos, Conflitos e Retrocessos

- Árvore parcial da busca:** a Tabela 18 exibe a ordem dos passos. A Figura 7 mostra uma representação visual da árvore parcial.
- Ordem de atribuição das variáveis:** $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_5 \rightarrow T_6$ (ordem fixa e sequencial).
- Conflitos encontrados:**
 - Passo 2: $T_2 = A$ viola a não-consecutividade com $T_1 = A$.
 - Passo 4: $T_3 = B$ viola a não-consecutividade com $T_2 = B$. Além disso, $T_3 = A$ é vedado pela restrição unária.
 - Passo 7: $T_5 = A$ viola a não-consecutividade com $T_4 = A$.
- Estados descartados:** os estados $\{T_2 = A\}$, $\{T_3 = B\}$ (com $T_3 = A$ descartado pela restrição unária antes mesmo de ser tentado) e $\{T_5 = A\}$ foram descartados por inconsistência.
- Retrocessos realizados:** nenhum retrocesso explícito foi necessário nesta execução, pois em cada nível havia sempre pelo menos um valor válido disponível.
- Solução final encontrada:** $T_1 = A, T_2 = B, T_3 = C, T_4 = A, T_5 = B, T_6 = A$

Table 18. Trace do Backtracking Simples

Passo	Variável atribuída	Status	Estado parcial
1	$T_1 = A$	OK	$\{T_1 = A\}$
2	$T_2 = A$	CONFLITO	$\{T_1 = A, T_2 = A\}$
3	$T_2 = B$	OK	$\{T_1 = A, T_2 = B\}$
4	$T_3 = B$	CONFLITO	$\{T_1 = A, T_2 = B, T_3 = B\}$
5	$T_3 = C$	OK	$\{T_1 = A, T_2 = B, T_3 = C\}$
6	$T_4 = A$	OK	$\{T_1 = A, T_2 = B, T_3 = C, T_4 = A\}$
7	$T_5 = A$	CONFLITO	$\{T_1 = A, T_2 = B, T_3 = C, T_4 = A, T_5 = A\}$
8	$T_5 = B$	OK	$\{T_1 = A, T_2 = B, T_3 = C, T_4 = A, T_5 = B\}$
9	$T_6 = A$	OK	$\{T_1 = A, T_2 = B, T_3 = C, T_4 = A, T_5 = B, T_6 = A\}$

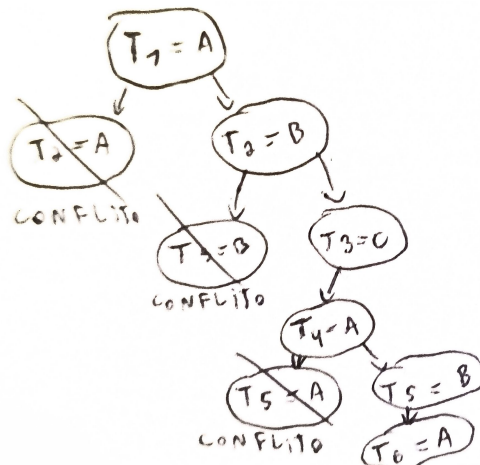


Figure 7. Árvore de busca CSP

4.3. Resolução com MRV e Degree Heuristic

Table 19. Trace do Backtracking com MRV e Degree Heuristic (passos seleccionados)

Passo	Variável atribuída	Estado parcial	Conflito detectado
1	$T_3 = B$	$\{T_3 = B\}$	—
2	$T_5 = A$	$\{T_3 = B, T_5 = A\}$	—
3	$T_1 = A$	$\{T_3 = B, T_5 = A, T_1 = A\}$	—
4	$T_2 = A$	$\{T_3 = B, T_5 = A, T_1 = A, T_2 = A\}$	não-consecutivo c/ $T_1 = A$
5	$T_2 = B$	$\{T_3 = B, T_5 = A, T_1 = A, T_2 = B\}$	não-consecutivo c/ $T_3 = B$
6	$T_2 = C$	$\{T_3 = B, T_5 = A, T_1 = A, T_2 = C\}$	—
7	$T_4 = A$	$\{T_3 = B, T_5 = A, T_1 = A, T_2 = C, T_4 = A\}$	não-consecutivo c/ $T_5 = A$
8	$T_4 = B$	$\{T_3 = B, T_5 = A, T_1 = A, T_2 = C, T_4 = B\}$	não-consecutivo c/ $T_3 = B$
9	$T_4 = C$	$\{T_3 = B, T_5 = A, T_1 = A, T_2 = C, T_4 = C\}$	—
<i>... retrocessos em T_6 e T_4 por falta de vizinho válido ...</i>			
12	$T_1 = B$	$\{T_3 = B, T_5 = A, T_1 = B\}$	—
13	$T_2 = A$	$\{T_3 = B, T_5 = A, T_1 = B, T_2 = A\}$	—
14	$T_4 = C$	$\{T_3 = B, T_5 = A, T_1 = B, T_2 = A, T_4 = C\}$	—
15	$T_6 = B$	$\{T_3 = B, T_5 = A, T_1 = B, T_2 = A, T_4 = C, T_6 = B\}$	Solução encontrada

- a) **Variável escolhida em cada passo:** Em vez de seguir a ordem fixa, a heurística MRV selecciona a variável com menor número de valores restantes no domínio.

Com a restrição unária já aplicada, T_3 possui apenas 3 valores ($\{B, C, D\}$) enquanto as demais têm 4. Logo, T_3 é escolhida primeiro (Passo 1). Após atribuir $T_3 = B$, os domínios de T_2 e T_4 perdem o valor B por não-consecutividade. Em seguida, o Degree Heuristic desempata a favor de T_5 (Passo 2), que tem dois vizinhos não atribuídos (T_4 e T_6). A sequência resultante foi: $T_3 \rightarrow T_5 \rightarrow T_1 \rightarrow T_2 \rightarrow T_4 \rightarrow T_6$.

- b) **Por que foi escolhida:** A variável T_3 foi selecionada primeiro porque possui o menor domínio (3 valores contra 4 das demais), o que caracteriza o critério MRV: minimizar o número de valores restantes. Variáveis com domínio menor têm maior probabilidade de falhar, e falhar cedo economiza trabalho.
- c) **Como MRV reduziu o espaço de busca:** Ao selecionar primeiro as variáveis mais restringidas, o MRV provoca falhas mais cedo na árvore, podando ramos inteiros antes de serem expandidos. Isso reduz o número de estados explorados em comparação com a ordem fixa.
- d) **Como Degree influenciou a busca:** O Degree Heuristic é usado como critério de desempate quando múltiplas variáveis possuem o mesmo tamanho de domínio. Variáveis com mais restrições sobre vizinhos não atribuídos são priorizadas, pois sua atribuição restringe mais o restante do problema e tende a revelar inconsistências mais rapidamente.

4.4. Forward Checking

- a) **Redução dos domínios após cada atribuição:** Após cada atribuição, o Forward Checking propaga a restrição de não-consecutividade, removendo o valor atribuído dos domínios dos vizinhos não atribuídos.

Table 20. Domínios após cada atribuição com Forward Checking

Passo	Atribuição	Podas realizadas
1	$T_3 = B$	Remove B de $D(T_2)$ e $D(T_4)$
2	$T_2 = A$	Remove A de $D(T_1)$
3	$T_1 = B$	Sem podas adicionais
4	$T_4 = A$	Remove A de $D(T_5)$
5	$T_5 = B$	Remove B de $D(T_6)$
6	$T_6 = A$	Solução completa

- b) **Domínios eliminados:** Os valores B foram eliminados de $D(T_2)$ e $D(T_4)$ após $T_3 = B$; A foi eliminado de $D(T_1)$ após $T_2 = A$; A de $D(T_5)$ após $T_4 = A$; e B de $D(T_6)$ após $T_5 = B$.
- c) **Momento em que inconsistências foram detectadas:** Nesta execução o Forward Checking encontrou a solução sem inconsistências (domínio vazio), pois a propagação guiou a busca diretamente a um caminho válido em apenas 6 atribuições.
- d) **Comparação com backtracking simples:** O Forward Checking explorou apenas 6 estados (contra 9 do Backtracking simples e sem retrocessos em ambos neste caso). A vantagem principal é a detecção antecipada de inconsistências: ao verificar que algum vizinho ficaria com domínio vazio após uma atribuição, o algoritmo aborta antes mesmo de tentar atribuições futuras que certamente falhariam.

4.5. Backjumping

- a) **Variáveis responsáveis pelos conflitos:** O Backjumping rastreia, para cada variável, quais variáveis anteriores causaram conflitos (conjunto de conflitos). Quando ocorre um *dead-end*, o algoritmo identifica a variável mais recente responsável pelo conflito.
- b) **Saltos realizados:** Nesta instância específica, devido à simplicidade do problema, o Backjumping comportou-se de forma equivalente ao Backtracking simples (9 estados, 0 saltos com retorno acima do último nível). O benefício do Backjumping torna-se mais visível em instâncias maiores, onde o culpado de um conflito está vários níveis acima na árvore, permitindo saltá-los.
- c) **Diferenças para backtracking tradicional:** No Backtracking tradicional, ao encontrar um *dead-end*, o algoritmo retrocede sempre ao nível imediatamente anterior. No Backjumping, o retrocesso vai diretamente à variável culpada pelo conflito, saltando variáveis intermediárias que não têm relação com o impasse. Isso reduz significativamente o número de retrocessos em problemas com alta esparsidade de restrições.
- d) **Redução observada na árvore de busca:** Para esta instância, o problema é pequeno e muito conectado, então a diferença não é expressiva. Em problemas com variáveis menos interconectadas, o Backjumping pode reduzir a árvore em ordens de magnitude, pois evita reexplorar ramos que não têm relação com o conflito encontrado.

4.6. Comparação entre os Algoritmos

Table 21. Comparação teórica entre os algoritmos de busca CSP

Critério	Backtracking	BT + MRV + Degree	Forward Checking	Backjumping
a) Estados explorados	Moderado (depende da ordem das variáveis)	Reduzido (heurísticas guiam a busca)	Mínimo (propagação elimina ramos inviáveis antecipadamente)	Semelhante ao BT em problemas pequenos; menor em grafos esparsos
b) Número de Retrocessos	Retrocede sempre ao nível imediatamente anterior	Menos retrocessos graças à melhor ordem de atribuição	Poucos ou nenhum; falhas detectadas antes das atribuições	Salta diretamente à variável culpada, evitando níveis intermediários
c) Velocidade de execução	Alta (sem overhead de heurística)	Moderada (custo extra na seleção da variável)	Mais lenta por atribuição (propagação de restrições a cada passo)	Alta; overhead apenas no rastreamento do conjunto de conflito
d) Consumo de memória	Mínimo (pilha de recursão)	Mínimo (sem cópias de domínio)	Maior (armazena cópias dos domínios para restauração)	Moderado (mantém conjuntos de conflito por variável)
e) Facilidade de implementação	Muito simples	Simples (adiciona funções de seleção)	Moderada (exige gestão de domínios e restauração)	Complexa (rastreamento de culpados e mecanismo de salto)
f) Eficiência prática	Boa em instâncias pequenas; degrada exponencialmente com o tamanho	Boa em geral; Degree resolve empates sem custo adicional significativo	Excelente em instâncias altamente restritas; detecta <i>dead-ends</i> cedo	Excelente em problemas com restrições esparsas; benefício cresce com a profundidade

4.7. Código Python

Olhar o arquivo `q4.py` para ver a implementação.

5. Questão 5 – Minimax e Poda Alpha-Beta

5.1. Conceitos Fundamentais do Algoritmo Minimax

- a) **Objetivo do algoritmo Minimax:** O Minimax é um algoritmo de busca utilizado em jogos de dois jogadores com informação perfeita. Seu objetivo é determinar a jogada ótima para o jogador MAX, assumindo que o oponente (MIN) também joga de forma ótima. O algoritmo percorre a árvore de jogo até as folhas e propaga os valores de utilidade em direção à raiz, garantindo que MAX maximize e MIN minimize os valores em seus respectivos níveis.
- b) **Diferença entre nós MAX e MIN:** Nós MAX representam estados nos quais o jogador (agente) deve escolher a ação. Nesses nós, seleciona-se o filho com o maior valor de utilidade. Já os nós MIN representam estados do oponente, que escolhe a ação de menor valor, pois busca minimizar o ganho do agente. A alternância entre níveis MAX e MIN modela a natureza adversarial do jogo.
- c) **Conceito de utilidade:** A utilidade (ou função de avaliação) é um valor numérico atribuído a um estado terminal da partida que quantifica quão bom aquele resultado é para o jogador MAX. Valores altos indicam vitória ou vantagem para MAX; valores baixos indicam desvantagem ou derrota. Na árvore apresentada, as utilidades nas folhas são $[3, 5, 6, 9, 1, 2, 0, -1, 7, 4, 5, 6]$.
- d) **Propagação de valores na árvore:** O algoritmo é recursivo. Nas folhas, retorna-se o valor de utilidade diretamente. Nos nós internos, o valor é calculado como o máximo (nós MAX) ou mínimo (nós MIN) dos valores dos filhos, e então propagado ao pai. Ao final, a raiz possui o valor minimax, que é a utilidade garantida ao jogador MAX sob jogo ótimo mútuo.
- e) **Hipótese de adversário perfeito:** O Minimax assume que MIN sempre escolhe a ação que minimiza a utilidade de MAX, ou seja, joga de forma perfeitamente racional, sempre escolhendo a melhor jogada a todo momento. Na prática, adversários reais podem cometer erros, o que torna essa decisão cautelosa demais.
- f) **Objetivo da Poda Alpha-Beta:** A Poda Alpha-Beta é uma otimização do Minimax que elimina ramos da árvore de busca que não podem influenciar a decisão final. Ela mantém dois parâmetros: α (melhor valor garantido para MAX no caminho atual) e β (melhor valor garantido para MIN no caminho atual). Quando $\alpha \geq \beta$, o ramo é podado. O objetivo é reduzir o número de nós avaliados sem alterar a decisão final, melhorando a eficiência computacional de $O(b^m)$ para $O(b^{m/2})$ no caso ótimo.

5.2. Execução Passo a Passo do Algoritmo Minimax

Para visualizar mais facilmente, eu nomeei cada nó seguindo a ordem alfabética, com ordem por nível e da esquerda para a direita. Com isso, tem-se os nós A (nível 0), B, C, D (nível 1), E, F, G, H, I, J (nível 2).

- a) **Valores calculados em cada nó:** a Tabela 22 mostra os valores de cada nó. A Figura 8 exibe visualmente o algoritmo. Segue a descrição texto:

Cálculo dos nós de nível MAX (profundidade 2):

$$E = \max(3, 5) = 5$$

$$F = \max(6, 9) = 9$$

$$G = \max(1, 2) = 2$$

$$H = \max(0, -1) = 0$$

$$I = \max(7, 4) = 7$$

$$J = \max(5, 6) = 6$$

Cálculo dos nós de nível MIN (profundidade 1):

$$B = \min(E, F) = \min(5, 9) = 5$$

$$C = \min(G, H) = \min(2, 0) = 0$$

$$D = \min(I, J) = \min(7, 6) = 6$$

Cálculo da raiz (nível MAX):

$$A = \max(B, C, D) = \max(5, 0, 6) = 6$$

Decisão de MAX: escolhe o filho **D** (valor 6). Caminho escolhido: $A \rightarrow D \rightarrow J$ (ou I , ambos explorados).

Table 22. Valores Minimax por nó

Nó	Tipo	Valor Minimax
A	MAX	6
B	MIN	5
C	MIN	0
D	MIN	6
E	MAX	5
F	MAX	9
G	MAX	2
H	MAX	0
I	MAX	7
J	MAX	6

b) **Ordem de expansão:** A, B, E, F, C, G, H, D, I, J.

c) **Propagação dos valores,** d) **Decisão tomada por Max,** e) **Caminho escolhido pelo algoritmo,** f) **Árvore parcialmente preenchida:** são encontrados na Tabela 22 e Figura 8.

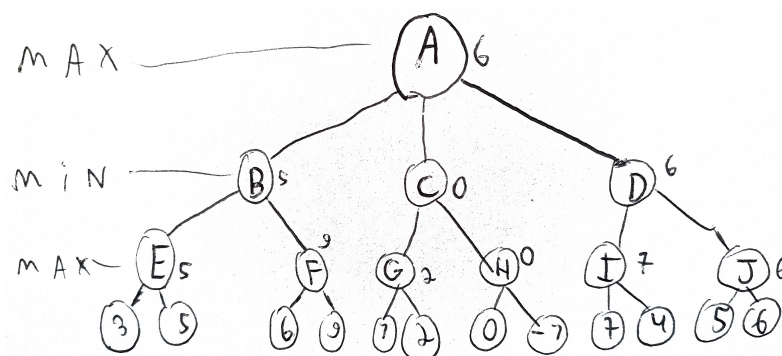


Figure 8. Árvore Minimax

5.3. Execução da Poda Alpha-Beta

- a) **Valores de α e β em cada passo:** apresentados na Tabela 23 e Figura 9.
- b) **Momentos em que ocorreram podas:** no passo 8, ao tentar expandir F, pois a primeira folha já mostrava $\alpha \geq \beta$. Também ao tentar expandir H, no passo 13.
- c) **Ramos podados:** nó folha com valor 9 e o nó **H** e seus dois valores terminais $[0, -1]$.
- d) **Motivo matemático da poda:** após avaliar G, o nó C atualiza $\beta = \min(+\infty, 2) = 2$. O nó A possui $\alpha = 5$. Como $\beta \leq \alpha$ ($2 \leq 5$), C nunca poderá retornar um valor maior que 2, que é inferior ao já garantido por A via B ($\alpha = 5$). Portanto, A jamais escolherá C, tornando desnecessária a avaliação de H. O mesmo princípio seguiu para a primeira poda, em que um nó folha não precisou ser avaliado pelo motivo de ser impossível de ser escolhido lidando com jogadas ideais.
- e) **Quantidade de nós não explorados:** 3 nós-folha não explorados, ocorreram 2 podas.

Table 23. Passo a passo da Poda Alpha-Beta

Passo	Nó	Tipo	α	β	Poda?
1	A	MAX	$-\infty$	$+\infty$	não
2	B	MIN	$-\infty$	$+\infty$	não
3	E	MAX	$-\infty$	$+\infty$	não
4	K	FOLHA	$-\infty$	$+\infty$	não
5	L	FOLHA	3	$+\infty$	não
6	F	MAX	$-\infty$	5	não
7	M	FOLHA	$-\infty$	5	não
8	N	FOLHA	6	5	SIM
9	C	MIN	5	$+\infty$	não
10	G	MAX	5	$+\infty$	não
11	O	FOLHA	5	$+\infty$	não
12	P	FOLHA	5	$+\infty$	não
13	H	—	5	2	SIM
—	Q	FOLHA	—	—	SIM
—	R	FOLHA	—	—	SIM
14	D	MIN	5	$+\infty$	não
15	I	MAX	5	$+\infty$	não
16	S	FOLHA	5	$+\infty$	não
17	T	FOLHA	7	$+\infty$	não
18	J	MAX	5	7	não
19	U	FOLHA	5	7	não
20	V	FOLHA	6	7	não

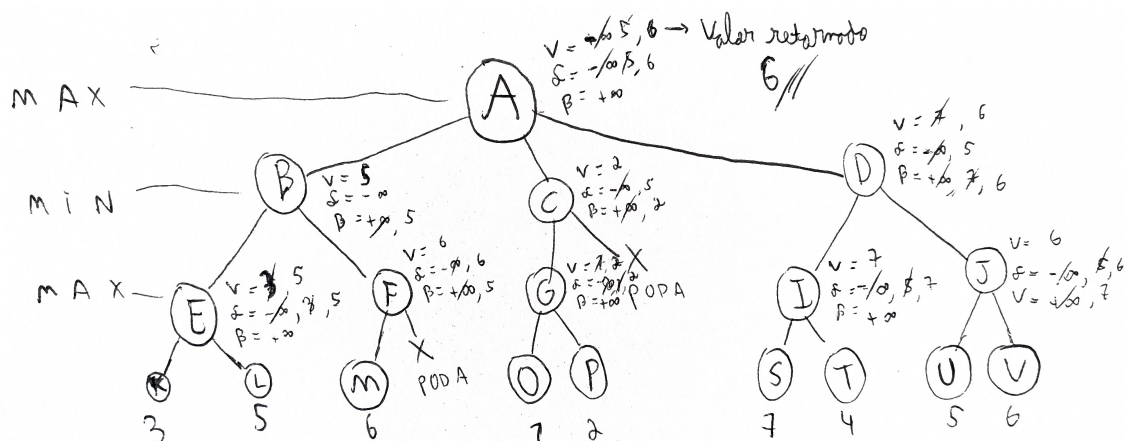


Figure 9. Árvore Poda Alfa-Beta

5.4. Ordenação dos Movimentos para Maximizar Podas

Para maximizar o número de podas, devemos expandir primeiro os movimentos que provavelmente serão os melhores: no nível MAX, os filhos com maior valor; no nível MIN, os filhos com menor valor. A nova ordem proposta é:

$$A : (D, B, C) \quad D : (J, I) \quad B : (E, F) \quad C : (H, G)$$

Com a ordem ótima:

- D: avalia $J=6$ ($\beta=6$), depois $I=7$ ($\beta=6$); retorna 6. $\alpha \leftarrow 6$.
- B: avalia $E=5$ ($\beta=5$). Como $\beta(5) \leq \alpha(6)$, poda F.
- C: avalia $H=0$ ($\beta=0$). Como $\beta(0) \leq \alpha(6)$, poda G.
- A retorna $\max(6, 5, 0) = 6$.

- Podas com a ordem original:** 2 podas, totalizando 3 nós-folha não explorados.
- Podas com a ordem ótima:** 2 (nós F e G), totalizando 4 nós-folha não explorados.
- Por que a nova ordem melhora a eficiência:** ao explorar primeiro o movimento de maior utilidade para MAX (D, valor 6), o limite α é estabelecido em um valor alto logo no início. Isso torna mais fácil que os nós MIN seguintes tenham $\beta \leq \alpha$, gerando podas antes.
- Relação entre ordenação e desempenho:** a Poda Alpha-Beta é muito dependente da ordem de expansão. Com a melhor ordenação possível, o algoritmo atinge complexidade $O(b^{m/2})$, efetivamente dobrando a profundidade de busca possível se comparado com o algoritmo sem a otimização. Com a pior ordenação, não há podas e o desempenho iguala o Minimax.

5.5. Minimax com Profundidade Limitada e Função Heurística

A busca é interrompida na profundidade 2 (nós E, F, G, H, I, J), substituindo a expansão real pela função heurística h :

$$h(E)=4, \quad h(F)=7, \quad h(G)=2, \quad h(H)=5, \quad h(I)=6, \quad h(J)=1$$

a) **Minimax limitado executado:**

$$B = \min(h(E), h(F)) = \min(4, 7) = 4$$

$$C = \min(h(G), h(H)) = \min(2, 5) = 2$$

$$D = \min(h(I), h(J)) = \min(6, 1) = 1$$

$$A = \max(4, 2, 1) = 4$$

b) **Valores heurísticos utilizados:** os seis valores acima são aplicados diretamente como estimativa do valor real dos estados.

c) **Comparação com o Minimax completo:** o Minimax completo retorna $A = 6$ e escolhe o filho D. O Minimax limitado retorna $A = 4$ e escolhe o filho B. As decisões **diferem**.

d) **Possíveis erros causados pela heurística:** a função heurística pode não refletir com precisão o valor real do estado. No exemplo, $h(J) = 1$ subestima o valor real de J (que vale 6 após expansão completa), levando D a ser avaliado como 1 em vez de 6, e revertendo a decisão. Erros heurísticos podem causar escolhas subótimas.

5.6. Comparação entre Minimax e Alpha-Beta

Table 24. Comparação entre Minimax e Alpha-Beta

Campo de Análise	Minimax	Alpha-Beta
a) Nós internos explorados	10	9
a) Valores terminais avaliados	12	9
a) Total de avaliações	22	18
b) Quantidade de podas	0	2
b) Nós não explorados	0	4 (H + 3 folhas)
c) Custo computacional	$O(b^m)$	$O(b^{m/2})$ (melhor caso)
d) Consumo de memória	$O(bm)$	$O(bm)$
e) Impacto da ordenação	Nenhum	Alto (mais podas)
f) Qualidade das decisões	Ótima (6, D)	Ótima (6, D)

Ambos os algoritmos produzem a mesma decisão ótima. A Poda Alpha-Beta é estritamente superior em custo computacional sem sacrificar qualidade.

5.7. Experimento: Modificação de Três Folhas

As seguintes folhas foram modificadas:

$$E : [3, 5] \rightarrow [10, 5], \quad F : [6, 9] \rightarrow [6, 1], \quad I : [7, 4] \rightarrow [3, 4]$$

Novos valores após Minimax:

$$E = \max(10, 5) = 10, \quad F = \max(6, 1) = 6, \quad G = 2 \text{ (não muda),}$$

$$H = 0 \text{ (não muda),} \quad I = \max(3, 4) = 4, \quad J = 6 \text{ (não muda),}$$

$$B = \min(10, 6) = 6, \quad C = \min(2, 0) = 0, \quad D = \min(4, 6) = 4,$$

$$A = \max(6, 0, 4) = 6$$

O valor final de A permaneceu 6, porém a **decisão mudou de D para B**. Na execução do Alpha-Beta modificado, ocorrem **2 podas** (H e J). A análise demonstra que:

Table 25. Comparação dos valores antes e depois das modificações

Nó	Tipo	Valor original	Valor modificado
A	MAX	6	6
B	MIN	5	6
C	MIN	0	0
D	MIN	6	4
E	MAX	5	10
F	MAX	9	6
G	MAX	2	2
H	MAX	0	0
I	MAX	7	4
J	MAX	6	6

- a) O Minimax retorna o mesmo valor ótimo (6), porém via caminho diferente ($A \rightarrow B$).
- b) O Alpha-Beta também retorna 6 ($A \rightarrow B$), com 2 podas (H e J).
- c) A decisão final mudou (de D para B), apesar de o valor ótimo ser o mesmo. Isso ocorre porque B se tornou tão bom quanto D era.
- d) O algoritmo apresenta sensibilidade às utilidades: pequenas alterações nos valores das folhas podem redirecionar o caminho escolhido, mesmo que o valor minimax da raiz se mantenha.

5.8. Código Python

Olhar o arquivo `q5.py` para ver a implementação.

6. Questão 06 – Monte Carlo Tree Search (MCTS)

O problema considera um tabuleiro Connect-4 simplificado 4×4 . O estado inicial é:

.	.	.	.
.	.	.	.
A	V	.	.
V	A	.	.

Onde V = vermelho e A = amarelo. As ações possíveis são c_1, c_2, c_3, c_4 . O jogador vermelho deve decidir qual coluna jogar.

6.1. Etapas do MCTS

- Seleção:** A partir da raiz, o algoritmo desce pela árvore escolhendo, em cada nó, o filho com maior valor UCT. Esse processo continua até encontrar um nó não totalmente expandido (com ações ainda não exploradas) ou um nó terminal. A UCT equilibra exploração (filhos com alta taxa de vitória) e exploração (filhos pouco visitados).
- Expansão:** Ao atingir um nó não totalmente expandido, uma das ações ainda não tentadas é selecionada e um novo nó filho é criado, representando o estado resultante dessa ação. O novo nó é inicializado com $N = 0$ e $W = 0$.
- Simulação (rollout):** A partir do nó recém-expandido, o jogo é simulado até o fim utilizando uma política de rollout (aleatória ou semi-gulosa). O resultado (vitória, derrota ou empate) é retornado como sinal de recompensa.
- Retropropagação:** O resultado do rollout é propagado de volta pelo caminho percorrido, da folha até a raiz. Em cada nó, $N(s)$ é incrementado em 1 e $W(s)$ é incrementado em 1 se o resultado foi vitória para o jogador considerado.

6.2. Papel das Estatísticas e Exploração vs. Exploração

- Papel do número de visitas $N(s)$:** Indica quantas vezes um nó foi visitado. Atua como denominador no cálculo da UCT, penalizando nós muito visitados no termo de exploração e conferindo maior confiança estatística à estimativa de valor quanto maior for N .
- Papel do número de vitórias $W(s)$:** Acumula as recompensas obtidas nas simulações que passaram pelo nó. A razão W/N fornece a taxa de vitória empírica e constitui o termo de exploração da UCT.
- Diferença entre exploração e exploração:** Exploração (termo w_j/n_j) privilegia nós com alta taxa de vitória já observada. Exploração (termo $C\sqrt{\ln N/n_j}$) privilegia nós pouco visitados. O parâmetro C controla o equilíbrio entre os dois: valores altos favorecem exploração; valores baixos favorecem exploração.

6.3. 10 Iterações do MCTS ($C = 1,4$, rollout aleatório)

Os resultados abaixo foram obtidos com semente aleatória fixa para reprodutibilidade.

- Caminho selecionado, b) nó expandido, c) resultado do rollout e d) atualização de $N(s)$ e $W(s)$** podem ser visualizado na tabela abaixo:

It.	Caminho	Nó expandido	Resultado	Raiz após it.	
				$N(\text{raiz})$	filhos
1	raiz	c_1	V vence	1	$c_1: N=1, W=1$
2	raiz	c_4	empate	2	$c_1: N=1, W=1; c_4: N=1, W=0$
3	raiz	c_2	empate	3	$c_2: N=1, W=0$ adicionado
4	raiz	c_3	empate	4	$c_3: N=1, W=0$ adicionado
5	raiz $\rightarrow c_1$	c_2	empate	5	$c_1: N=2, W=1$
6	raiz $\rightarrow c_4$	c_2	V vence	6	$c_4: N=2, W=1$
7	raiz $\rightarrow c_2$	c_2	empate	7	$c_2: N=2, W=0$
8	raiz $\rightarrow c_3$	c_1	empate	8	$c_3: N=2, W=0$
9	raiz $\rightarrow c_1$	c_1	V vence	9	$c_1: N=3, W=2$
10	raiz $\rightarrow c_4$	c_1	empate	10	$c_4: N=3, W=1$

- e) **Árvore parcial após 10 iterações:** A raiz possui quatro filhos diretos (c_1 a c_4). Cada filho contém entre 1 e 3 nós neto. O estado da raiz é $N = 10$, W acumulado = 4.

6.4. Cálculo da UCT ($C = 1,4$)

Após 10 iterações, com $N = 10$ (visitas da raiz):

- a) **Valor de UCT para cada filho:**

Jogada	Visitas	Vitórias	Exploração	UCT
c_1	3	2	0,6667	1,8932
c_2	2	0	0,0000	1,5022
c_3	2	0	0,0000	1,5022
c_4	3	1	0,3333	1,5599

- b) **Nó selecionado:** c_1 , com o maior valor UCT de 1,8932.
c) **Influência do termo de exploração:** Para c_2 e c_3 ($N=2$), o termo de exploração é $1,4\sqrt{\ln 10/2} \sim 1,502$, representando 100% do UCT, pois a taxa de vitória é zero.
d) **Influência do termo de exploração:** Para c_1 ($N=3$, $W=2$), a taxa de vitória $2/3 \sim 0,667$ acrescenta 0,667 ao UCT, distinguindo-o de c_4 (taxa 0,333) mesmo com o mesmo número de visitas.

6.5. Comparação de Valores de C (200 iterações)

Critério	$C = 0,1$	$C = 1,4$	$C = 3,0$
Melhor jogada encontrada	c_1	c_4	c_2
Visitas c_1 (frac.)	194 (97%)	57 (28%)	47 (24%)
Visitas c_2 (frac.)	2 (1%)	36 (18%)	63 (32%)
Visitas c_3 (frac.)	2 (1%)	24 (12%)	41 (21%)
Visitas c_4 (frac.)	2 (1%)	83 (42%)	49 (25%)
a) Quantidade de exploração	Muito baixa	Equilibrada	Muito alta
b) Diversidade de jogadas	Muito baixa	Moderada	Alta
c) Estabilidade das decisões	Alta (não confiável)	Moderada	Baixa
d) Qualidade das jogadas	Pobre (vicia em c_1)	Boa	Pobre (dilui recursos)

6.6. Comparação de Rollouts (200 iterações, $C = 1,4$)

- a) **Qualidade das decisões:** O rollout semi-guloso produz estimativas de valor mais acuradas pois prioriza jogadas centrais e bloqueia vitórias imediatas, gerando simulações mais representativas do jogo real.
- b) **Velocidade de convergência:** O rollout semi-guloso converge para c_2 em todas as contagens de iterações testadas (10 a 500). O rollout aleatório oscila entre c_1 e c_4 .
- c) **Número de simulações necessárias:** O rollout aleatório requer substancialmente mais simulações para produzir uma decisão estável, enquanto o semi-guloso converge com ~ 25 iterações.

As informações detalhadas da comparação entre rollout aleatório e rollout semi-guloso podem ser vistas na tabela abaixo:

Critério	Aleatório	Semi-guloso
Melhor jogada	c_4	c_2
Taxa de vitória c_1	0,368	0,553
Taxa de vitória c_2	0,250	0,745
Taxa de vitória c_3	0,125	0,417
Taxa de vitória c_4	0,446	0,600
Convergência estabiliza em	500 it. (instável)	10 it.
Simulações para decisão confiável	>500	~ 25
a) Qualidade das decisões	Moderada	Alta
b) Velocidade de convergência	Lenta	Rápida
c) Número de simulações necessárias	Alto	Baixo

6.7. Comparação Final

Critério	Aleatório, $C=1,4$	Semi-guloso, $C=1,4$	Aleatório, $C=0,1$	Aleatório, $C=3,0$
a) Rollout aleatório vs semi-guloso	Estimativas ruidosas, $WR \sim 0,37-0,45$	Estimativas precisas, $WR \sim 0,55-0,75$	—	—
b) Diferentes valores de C	c_4 (42% das visitas)	c_2 (49% das visitas)	c_1 (97% das visitas, vicia)	c_2 (32% das visitas, uniforme)
c) Número de iterações	Instável até 500 iterações	Estável a partir de 10 iterações	Estável mas enviesado	Oscila
d) Estabilidade das decisões	Baixa	Alta (c_2 em todos os testes)	Alta mas incorreta	Baixa

Em resumo, o rollout semi-guloso com $C = 1,4$ apresenta o melhor desempenho geral: converge rapidamente para c_2 , que bloqueia uma possível vitória do adversário, e produz estimativas de valor significativamente mais confiáveis do que o rollout puramente aleatório.

6.8. Código Python

Olhar o arquivo `q6.py` para ver a implementação.